

Part 1: The Phantom Bug

- Write a program that tosses 50,000,000 random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.
- **Deliverable:** Run it 5 times. Record the value of π .
- *The Lesson:* The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race).

Part 2: The Synchronization Trap

- Fix the race condition by adding `synchronized` to the increment or using `AtomicLong.incrementAndGet()`.
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for $T \in \{1, 2, 4, 8, 16, 32\}$ threads on their machine with 100,000,000 iterations.

- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (TN T1)	Efficiency (TSpeedup)
1	<i>(baseline)</i>	1.0x	100%
2	...	...	...
4	...	...	...
8	...	...	...
16	...	...	...

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?
2. Why was the synchronized version in Part 2 slower than running on one single core?

Upload your solutions before the class end-time to the Git repo that you created for benchmarks. Share repo access with @sufyanism (GitHub/GitLab).

Run	π value
1	2.48892928
2	2.64365240
3	2.53445192
4	2.55107096
5	2.52626888

Version	π	Runtime
Synchronized	3.14124136	4556 ms

8	532	5.74x	71.80%
16	533	5.73x	35.83%
32	514	5.95x	18.58%

Questions

1. Why didn't the 8-core CPU run twice as fast with 16 threads compared with 8 threads?

At 8 threads, the program can effectively use the 8 CPU cores of the M2 processor. When the number of threads increases to 16, there are more threads than CPU cores, so the operating system has to schedule multiple threads on the same cores. This creates additional scheduling overhead, so the runtime stayed almost the same: 532 ms with 8 threads and 533 ms with 16 threads.

2. Why was the synchronized version slower than one single-threaded loop?

The synchronized version was slower because all threads continuously accessed the same shared `totalHits` variable. Every increment had to enter a synchronized section, so the threads spent time waiting for access to the shared resource. This synchronization overhead reduced the benefit of using multiple threads. In contrast, the reduction version uses a separate local counter for each thread and combines the results only at the end.

Threads (T)	Runtime (ms)	Speedup	Efficiency
1	3056	1.00x	100.00%
2	1605	1.90x	95.20%
4	868	3.52x	88.02%